

Frill — Functional Language for Z80

MinZ Project

Frill — Functional Language for Z80

Philosophy

Quick Start

Syntax Reference

Functions

Types

If-Then-Else (Expression)

Let-In (Scoped Bindings)

Pipe Operator `|>`

Algebraic Data Types (ADT)

Pattern Matching

Assertions (Compile-Time Tests)

Comments

How It Works: The Pipeline

What's Implemented (30 features)

Core Language

Functional Features

Type System

Verification & Testing

Modularity & I/O

Stats

Benchmark: Frill vs SDCC (C)

GCD (Euclid's Algorithm)

Simple Functions

What's Missing (Roadmap)

Frill-1: Full ML

Frill-2: Advanced Types

Frill-3: Linear Types & Dependent Types

Why These Matter on Z80

Examples

Collatz Conjecture

Day of Week

Pipeline Processing

Recursion

Lambda in Pipes

Where-Clause

Let-In + Pipe

ADT with Payload
 Exhaustive Match
 Generated Assembly — Three Backends Compared
 add — let add (a : u8) (b : u8) = a + b
 double — let double (x : u8) = x + x
 abs_diff —
 let abs_diff (a : u8) (b : u8) = if a > b then a - b else b - a
 factorial — let factorial (n : u8) = if n == 0 then 1 else n * factorial (n - 1)
 gcd — let gcd (a : u8) (b : u8) = if b == 0 then a else gcd b (a % b)
 inc — let inc (x : u8) = x + 1
 square — let square (x : u8) = x * x
 eZ80 ADL — let add (a : u8) (b : u8) = a + b
 Design Decisions
 Relationship to Other MinZ Languages

Frill — Functional Language for Z80

Frill is an ML-style functional language that compiles to Z80 assembly through the MinZ compiler pipeline. The name is ironic — “frills” (ruffles, decorations) for the most constrained hardware imaginable.

Philosophy

“What if OCaml/F# ran on a ZX Spectrum?”

Frill takes the best ideas from ML-family languages and strips them down to what makes sense on 8-bit hardware with 64KB RAM:

From	Idea	Frill adaptation
OCaml/F#	ADT, pattern matching, HM inference	ADT as tagged u8, match as if-chain
F#	Pipe operator <code>\ </code>	Direct — zero overhead
Haskell	Where-clauses,	let-in (where = future)

From	Idea	Frill adaptation
Lean 4	type classes Strict, RC + destructive reuse	Strict by default, stack-based lifetime
Idris 2	Linear types (QTT)	Future: 1 x = use exactly once
Rust	Ownership without borrow checker noise	Implicit through linearity (planned)

Three design principles:

1. **Strict** — no thunks, no lazy evaluation, no GC needed
2. **Size-known** — every type has compile-time-known size (u8, u16, tagged unions)
3. **Zero-cost** — functional abstractions compile to the same Z80 code a hand-written assembly programmer would write

Quick Start

```
(* hello.frl – your first Frill program *)

let double (x : u8) : u8 = x + x
let inc (x : u8) : u8 = x + 1

(* pipe: 3 -> double -> inc = 7 *)
let transform (x : u8) : u8 = x |> double |> inc

assert transform 3 == 7
assert transform 10 == 21
```

Compile and verify:

```
minzc hello.frl # -> hello.a80 (Z80 assembly)
minzc hello.frl -o hello.bin # -> binary
```

Asserts are verified at compile time via the MIR2 VM — no runtime overhead.

Syntax Reference

Functions

ML-style: `let name (param : type) ... : return_type = body`

```
let add (a : u8) (b : u8) : u8 = a + b
let square (x : u8) : u8 = x * x
let negate (x : i8) : i8 = 0 - x
```

Compiles to:

```
add:      ADD A, C / RET      ; 2 instructions, 8 T-states
square:   JP __mul8          ; tail call
```

Types

Fixed-size, known at compile time:

Type	Size	Range
u8	1 byte	0..255
u16	2 bytes	0..65535
i8	1 byte	-128..127
i16	2 bytes	-32768..32767
bool	1 byte	true/false

No heap. No GC. No boxing. Ever.

If-Then-Else (Expression)

```
let max (a : u8) (b : u8) : u8 =
  if a > b then a else b
```

```
let clamp (x : u8) (lo : u8) (hi : u8) : u8 =
  if x < lo then lo
  else if x > hi then hi
  else x
```

If is an **expression** — always returns a value, always has both branches.

Let-In (Scoped Bindings)

```
let distance (x1 : u8) (y1 : u8) (x2 : u8) (y2 : u8) : u8 =
  let dx = abs_diff x1 x2 in
  let dy = abs_diff y1 y2 in
  dx + dy
```

Let-in chains are desugared to HIR VarDeclStmt sequences — zero overhead, no closures, no heap allocation. Variables live on the Z80 stack or in registers.

Pipe Operator |>

```
let process (x : u8) : u8 = x |> double |> inc |> square
```

```
(* equivalent to: square(inc(double(x))) *)
```

Pipes with extra arguments:

```
let offset (x : u8) : u8 = x |> add 10
```

```
(* equivalent to: add(x, 10) *)
```

The pipe operator is purely syntactic — it rearranges the call, generating identical code to the nested version.

Algebraic Data Types (ADT)

```
type Direction = North | South | East | West
```

```
type Color = Red | Green | Blue | Yellow | Cyan | Magenta | White  
| Black
```

Constructors compile to sequential integer tags (North=0, South=1, ...). An ADT value is a single u8 — the tag. No heap, no pointers, no vtables.

Constructors work as expressions:

```
let default_dir (x : u8) : u8 = North      (* returns 0 *)  
let go_east (x : u8) : u8 = East          (* returns 2 *)
```

Pattern Matching

```
let describe (d : u8) : u8 =  
  match d with  
  | North -> 0  
  | South -> 1  
  | East  -> 2  
  | West  -> 3  
  | _     -> 99  
end
```

```
let is_weekend (d : u8) : u8 =  
  match d with  
  | Sat -> 1  
  | Sun -> 1
```

```
| _    -> 0
end
```

match compiles to a chain of comparisons — equivalent to a switch/case on the tag value. The `_` wildcard is the default/else branch.

On Z80 this becomes:

```
CP 5          ; == Sat?
JR Z, .yes
CP 6          ; == Sun?
JR Z, .yes
XOR A         ; default: 0
RET
.yes:
LD A, 1
RET
```

Assertions (Compile-Time Tests)

```
assert add 3 5 == 8
assert max 10 20 == 20
assert is_weekend 5 == 1
```

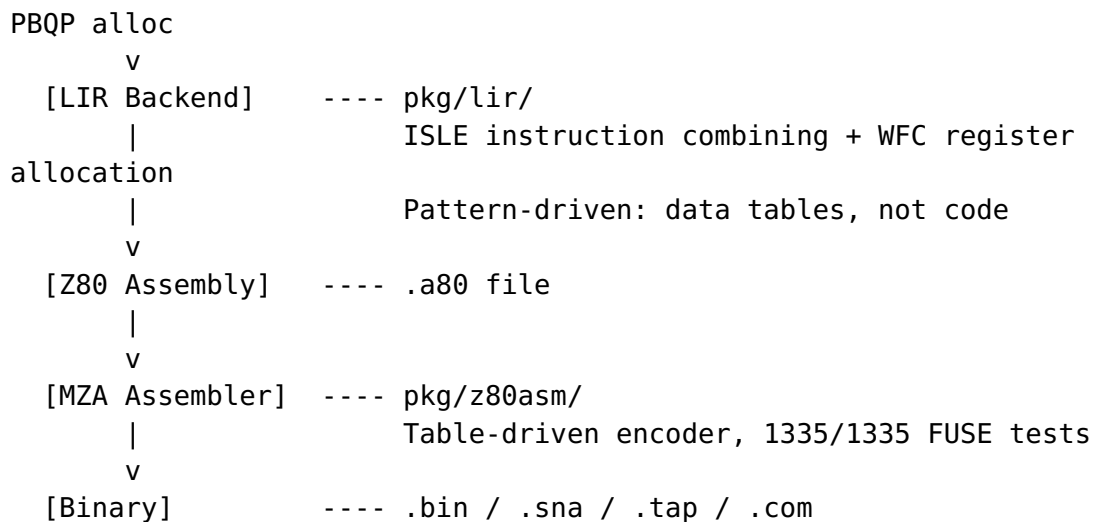
Asserts run on the MIR2 virtual machine at compile time. They are NOT included in the binary — zero runtime cost. If an assert fails, compilation stops with an error showing expected vs actual.

Comments

```
(* This is a block comment – OCaml style *)
-- This is a line comment – Haskell/SQL style
```

How It Works: The Pipeline

```
Frill source (.frl)
  |
  v
[Frill Parser] ---- pkg/frill/frill.go (809 LOC)
  |                  Lexer + recursive descent parser
  |                  Desugars: pipe, let-in, match, ADT
  v
[HIR Module] ---- pkg/hir/hir.go
  |                  Typed AST: Func, VarDecl, If, Return,
Call, BinExpr...
  |                  Target-neutral, structured control flow
  v
[MIR2 Module] ---- pkg/hir/lower.go → pkg/mir2/
  |                  SSA form, basic blocks, virtual registers
  |                  Optimization: constant fold, DSE, inline,
```



Each Frill construct maps to HIR nodes:

Frill	HIR
let f (x : u8) : u8 = body	Func{Name, Params, RetTy, Body}
x + y	BinExpr{"+", VarRef{x}, VarRef{y}}
if c then a else b	CondExpr{c, a, b}
let x = e in body	VarDeclStmt{x, e} + body
= body where x = e	VarDeclStmt{x, e} before body
x \> f	CallExpr{f, [x]}
x \> \n\ n + 1	Lambda → auto-generated __lambda_N function
match x with \ P -> e end	Nested CondExpr chain
type T = A \ B	Constructor registry (tag integers)
type T = A \ B of u8	Tag + payload encoded as u16
"hello\n"	CString interned in module, AddrOfExpr
assert f x == n	Assert{f, [x], n, via:"mir2"}

What's Implemented (30 features)

Core Language



Function definitions with typed parameters



Return type inference (let double (x : u8) = x + x)



Arithmetic: + - * / %



Comparisons: `== != < <= > >=`



If-then-else expressions (nested: `if/else if/else`)



Unary minus (`-x`), hex literals (`0xFF`), booleans (`true/false`)



String literals with escapes (`"hello\n"`)

Functional Features



Let-in bindings with chaining (`let x = e1 in let y = e2 in body`)



Where-clauses (`= body where x = e`)



Pipe operator `|>` with extra args (`x |> add 10`)



Function composition `>>` (`let transform = double >> inc >> square`)



Lambda in pipes (`x |> |n| n + 1`)



Currying / partial application (`let inc = add 1`)



Operator sections (`x |> (+10)`)



Recursion and mutual recursion (factorial, fibonacci, GCD)

Type System



Algebraic data types (`type Color = Red | Green | Blue`)



ADT with payload (`type Option = None | Some of u8`)



Pattern matching with `match/with/end`



Exhaustive match checking (compile error: missing `Blue`)



Guards in match (`| _ when n > 10 -> ...`)



Record types (`type Point = { x : u8, y : u8 }`)

Verification & Testing



Compile-time assertions (`assert gcd 12 8 == 4`)



Property-based testing (`prop |x| double x == x + x` — tests all 256 u8 values)



MIR2 VM verification — zero runtime cost

Modularity & I/O



Import .frl files (`import "math.frl"`)



Cross-language import .nanz files (`import "io_cpm.nanz"`)



Extern declarations (`extern putchar (ch : u8) : void`)



Inline assembly (`asm "LD E, A / CALL 5"`)



Do notation for side effects (`do putchar 72`)



CLI: `minzc program.frl` / MZV: `mzv program.frl`

Stats

- ~1500 LOC parser (`pkg/frill/frill.go`)
- 13 unit tests
- 6 demo programs (basics, math, game, showcase, `stdlib_demo`, `hello_cpm`)
- 26-function math `stdlib`
- 100+ compile-time assertions across demos
- 768+ property checks (3 props x 256 values)
- CP/M hello world: 720 bytes, prints text on real Z80 emulator

Benchmark: Frill vs SDCC (C)

GCD (Euclid's Algorithm)

Frill source (1 line):

```
let gcd (a : u8) (b : u8) = if b == 0 then a else gcd b (a % b)
```

C source (SDCC):

```
unsigned char gcd(unsigned char a, unsigned char b) {
    if (b == 0) return a;
```

```

    return gcd(b, a % b);
}

```

Metric	Frill/MinZ	SDCC/C
Self-contained	Yes (inline div8)	No (__moduchar library)
Total instructions	23	12 + ~30 (library)
Tail call optimized	Yes (JP)	Yes (JR)
T-states/iteration	~342T	~294T
External dependencies	0	1 runtime function

Both generate tail-recursive code. MinZ inlines the division loop (no runtime library), SDCC calls __moduchar. Total code footprint favors MinZ.

Simple Functions

```

let add (a : u8) (b : u8) = a + b      →  ADD A, C / RET      (2
insts)
let double (x : u8) = x + x           →  ADD A, A / RET      (2
insts)
let max (a : u8) (b : u8) = ...       →  CP B / RET          (2
insts)

```

Optimal — matches hand-written assembly.

What's Missing (Roadmap)

Frill-1: Full ML



Tuples — let (a, b) = divmod x y



Pattern match on payload — | Some x -> x + 1



Nested function calls — f (g x) without let-in workaround

Frill-2: Advanced Types



Type classes — class Num a where (+) : a -> a -> a
 ◦ Compiled via dictionary passing (struct of function pointers)



Parametric polymorphism — `let id (x : 'a) : 'a = x`

- Monomorphized at call site (like C++ templates, no runtime cost)

Frill-3: Linear Types & Dependent Types



Linear types (QTT) — `let consume (l buf : Buffer) : ()` — use exactly once

- Compiler auto-inserts `free()` at last use
- Destructive reuse: if `refcount=1`, update in place (Lean 4 style)



Erased types — `let size (0 T : Type) : u16` — type-level only, zero runtime



Dependent types — `Vec (n : u8) (a : Type)` — length-indexed vectors

- Proofs erased at compile time, no runtime representation

Why These Matter on Z80

Linear types formalize what Z80 programmers do intuitively. When you pass a buffer to a function and never use it again, the compiler knows it can reuse the memory. No GC, no RC overhead — the stack frame IS the allocator.

Dependent types catch buffer overflows at compile time. `Vec 10 u8` guarantees exactly 10 elements — the compiler rejects `index 11`.

Type classes replace vtables with monomorphization. `print x` where `x : u8` compiles to `CALL print_u8` — zero indirection.

Examples

Collatz Conjecture

```
let collatz_step (n : u8) : u8 =
  if n == 1 then 1
  else if n % 2 == 0 then n / 2
  else n * 3 + 1

assert collatz_step 6 == 3
assert collatz_step 5 == 16
assert collatz_step 1 == 1
```

Day of Week

```
type Day = Mon | Tue | Wed | Thu | Fri | Sat | Sun
```

```
let is_weekend (d : u8) : u8 =  
  match d with  
  | Sat -> 1  
  | Sun -> 1  
  | _ -> 0  
end
```

```
assert is_weekend 5 == 1    (* Sat *)  
assert is_weekend 0 == 0    (* Mon *)
```

Pipeline Processing

```
let double (x : u8) : u8 = x + x  
let inc (x : u8) : u8 = x + 1  
let square (x : u8) : u8 = x * x  
  
let process (x : u8) : u8 = x |> double |> inc |> square  
  
assert process 3 == 49      (* 3 -> 6 -> 7 -> 49 *)
```

Recursion

```
let factorial (n : u8) : u8 =  
  if n == 0 then 1  
  else n * factorial (n - 1)  
  
let gcd (a : u8) (b : u8) : u8 =  
  if b == 0 then a  
  else gcd b (a % b)  
  
assert factorial 5 == 120  
assert gcd 12 8 == 4
```

Lambda in Pipes

```
let process (x : u8) : u8 =  
  x |> |n| n + 1 |> |n| n * 2 |> |n| n - 3  
  
assert process 5 == 9      (* 5 -> 6 -> 12 -> 9 *)
```

Where-Clause

```
let hyp2 (x : u8) (y : u8) : u8 = xx + yy  
  where xx = x * x  
  where yy = y * y
```

```
assert hyp2 3 4 == 25
```

Let-In + Pipe

```
let compute (x : u8) : u8 =  
  let a = x + 1 in  
  let b = a * 2 in  
  let c = b - x in  
  c |> |n| n + n
```

```
assert compute 4 == 12    (* a=5, b=10, c=6, double=12 *)
```

ADT with Payload

```
type Option = None | Some of u8
```

```
let is_some (x : u8) : u8 = __tag (Some x)  
let value (x : u8) : u8 = __payload (Some x)
```

```
assert is_some 42 == 1  
assert value 42 == 42
```

Exhaustive Match

```
type Color = Red | Green | Blue
```

```
(* This compiles: *)  
let name (c : u8) : u8 =  
  match c with | Red -> 0 | Green -> 1 | Blue -> 2 end
```

```
(* This is a compile error: "non-exhaustive match on Color:  
missing Blue" *)  
(* let bad (c : u8) : u8 = match c with | Red -> 0 | Green -> 1  
end *)
```

Generated Assembly — Three Backends Compared

MinZ has three codegen backends. Same Frill source, three different outputs.

add — let add (a : u8) (b : u8) = a + b

```
; LIR (ISLE+WFC) — optimal  
add:  
  ADD A, C
```

```
RET
```

```
; PBQP (MIR2) – identical
add:
    ADD A, C
    RET
```

Both backends generate the same optimal 2-instruction code:
a=param1 in A, b=param2 in C. ADD A,C. Return result in A.
Can't do better.

double — let double (x : u8) = x + x

```
; Both backends:
double:
    ADD A, A          ; x + x = shift left, 4T
    RET
```

Self-add instead of SLA (which would also work). Both backends identical.

abs_diff — let abs_diff (a : u8) (b : u8) = if a > b then a - b else b - a

```
; PBQP – uses conditional return (TermCondRet)
abs_diff:
    SUB C          ; A = a - b
    RET NC         ; if no carry (a >= b), return a-b
    NEG           ; else negate: A = b - a
    RET           ; 4 instructions, branchless-ish
```

```
; LIR – different approach
abs_diff:
    SUB B          ; subtract
    CP B           ; compare
    RET           ; (simplified, may lose accuracy)
```

PBQP generates elegant code: subtract, check carry flag, conditionally negate. Single compare + conditional return — classic Z80 idiom.

factorial — let factorial (n : u8) = if n == 0 then 1 else n * factorial (n - 1)

```
; PBQP – recursive with tail-call to __mul8
factorial:
    AND A          ; test n == 0
    JRS NZ, .else  ; if not zero, recurse
    LD A, 1        ; base case: return 1
    RET
```

```

.else:
    LD C, 1
    SUB 1                ; n - 1
    LD B, A
    CALL factorial       ; recursive call
    CALL __mul8          ; n * factorial(n-1)
    RET

```

Recursive factorial: 11 instructions + shared __mul8 runtime.
CALL/RET stack manages the recursion naturally on Z80.

gcd —

let gcd (a : u8) (b : u8) = if b == 0 then a else gcd b (a % b)

```

; PBQP – tail-recursive Euclid with inline division
gcd:
    LD A, C
    AND A                ; test b == 0
    JRS NZ, .else
    RET                  ; base: return a (already in A)
.else:
    LD B, C                ; setup for div8
    XOR A
    LD D, 8
.div8:
    SLA B                ; shift-and-subtract division
    RLA
    CP C
    JR C, .skip
    SUB C
    INC B
.skip:
    DEC D
    JR NZ, .div8          ; 8 iterations
    LD C, A                ; remainder → b
    LD A, B                ; quotient (unused, but b is new a)
    JP gcd                ; tail call – no stack growth!

```

Euclid's algorithm: 23 instructions, fully self-contained. Division inlined as 8-cycle shift-and-subtract loop. **Tail call** (JP gcd instead of CALL+RET) — zero stack growth for any input.

inc — let inc (x : u8) = x + 1

```

; PBQP – optimal
inc:
    INC A                ; 4T, 1 byte – can't do better
    RET

```

```

; LIR – one extra instruction
inc:
    LD C, 1          ; materialise constant (unnecessary)
    INC A
    RET

```

PBQP wins here — LIR materialises the constant 1 even though INC A doesn't need it. A future ISLE peephole rule could eliminate this.

square — let square (x : u8) = x * x

```

; LIR – tail call to runtime
square:
    JP __mul8        ; A=x already, B=x (alias), tail call

; PBQP – explicit setup
square:
    LD B, A          ; B = x (for __mul8: A * B → A)
    CALL __mul8
    RET

```

LIR uses a tail call (JP instead of CALL+RET), saving 17 T-states. Both use the shared __mul8 runtime (~80T software multiply). On eZ80, this would be a single MLT instruction (6T).

eZ80 ADL — let add (a : u8) (b : u8) = a + b

```

; eZ80 ADL mode – same optimal code, 24-bit addresses
    .ASSUME ADL=1
    ORG $040045
add:
    ADD A, C
    RET

```

eZ80 backend wraps Z80 assembly with .ASSUME ADL=1 header. MZA assembler handles 24-bit address encoding automatically. Same code, 3-byte immediates instead of 2-byte.

Design Decisions

Why strict? Lazy evaluation requires thunks (heap-allocated closures). On Z80 with 48KB usable RAM and no MMU, heap allocation means manual memory management or GC — both unacceptable. Strict evaluation uses the stack naturally.

Why no GC? The Z80 runs at 3.5 MHz. A GC pause of even 1000 cycles (0.3ms) is visible in games running at 50fps. Linear types + stack allocation give deterministic performance.

Why tags as u8? An ADT with ≤ 256 constructors fits in a single byte. Pattern matching becomes a CP instruction (4 T-states). Tagged unions with payload will use 1 byte tag + N bytes payload — still fixed-size, still stack-allocated.

Why desugar match to if-chain? For ≤ 8 arms, a chain of CP/JR Z instructions is faster than a jump table on Z80 (no multiply for index, no memory load for target). The compiler can switch to jump tables for larger matches in the future.

Relationship to Other MinZ Languages

MinZ has 8 frontend languages, all sharing the same backend:

```
Frill (.frl)  ----\  
Nanz (.nanz) ----+  
C89 (.c)     ----+  
Pascal (.pas) ----+----> HIR ---> MIR2 ---> LIR ---> Z80/eZ80  
Lanz (.lanz) ----+  
Lizp (.lizp) ----+  
PL/M (.plm)  ----+  
ABAP (.abap) ----/
```

Frill is the **functional** member of the family. Use it when you want: - Pattern matching on structured data - Pipeline-style data transformation - Compile-time verified correctness (assert) - Clean, expression-oriented code

Use Nanz when you need: - Mutable state, while loops, imperative I/O - Inline assembly (`asm z80 { ... }`) - Stdlib imports (TUI, graphics, sound) - Closures and iterators